



P113-Accessibility data warehousing and transformation system

T14A-Bread, Project Proposal for COMP[39]900

NAME	ZID	EMAIL	ROLE
Lu Zhang	z5530133	z5530133@ad.unsw.edu.au	project management
Han Bao	z5549732	z5549732@ad.unsw.edu.au	backend development
Yuchen wang	z5511989	z5511989@ad.unsw.edu.au	frontend development/Testing
Chi Man Fu	z5586971	z5586971@ad.unsw.edu.au	backend development
Zhaoyuan Xu	z5615760	z5615760@ad.unsw.edu.au	frontend development

Computer Science Project

27th April 2026

Content

1. Installation Manual.....1

2. System Architecture Diagram3

3. Design Justifications4

5. Limitations and future work..... 21

6. Handover 23

1. Installation Manual

Quick Summary

Prerequisite: Docker Desktop is installed on host platform

Build steps: At project root directory, please issue three commands for clean build

- a. docker compose down
- b. docker compose build --no-cache
- c. docker compose up -d

Run Application: <http://localhost:3333>

Backend API doc: <http://localhost:8000/docs>

Brief overview

The application is fully containerised using Docker, and no manual installation of Python, Node.js, MongoDB, or Redis is required. List of containers and ports used:

frontend	React + Vite UI	3333
backend	FastAPI API server	8000
worker	Schedule download	-
mongo	MongoDB database	27017
redis	Redis message queue	6379

All services are defined inside docker-compose.yml.

Environment Variables

No .env files are required. All environment variables are already defined inside our docker-compose.yml:

- a. backend environment: MONGO_URL=mongodb://mongo:27017, REDIS_URL=redis://redis:6379/0;
- b. worker uses the same configuration internally.

Secrets: There are no secrets or API keys required for this project.

Optional: make clean

If make utility is available on the platform, 'make clean' at project root can also execute the 3 build commands for clean build. The commands are defined inside Makefile at project root.

T14A-Bread project Test

Run backend tests inside the Docker container:

```
# docker build
docker compose up --build -d

# Install test dependencies
docker compose exec backend pip install pytest httpx

# Run all tests
docker compose exec backend pytest tests/ -v

# Run with coverage
docker compose exec backend pip install pytest-cov
docker compose exec backend pytest tests/ -v --cov=app
```

Run frontend tests :

```
# docker build
docker compose up --build -d

# cypress is not installed in the docker
# cd to /frontend and open another terminal
# install cypress first
npm install

# Run all tests
npm run cy:run

# Run with browser
npm run cy:open
```

Frontend tests detail is in frontend/cypress/e2e_test_summary.md

2. System Architecture Diagram

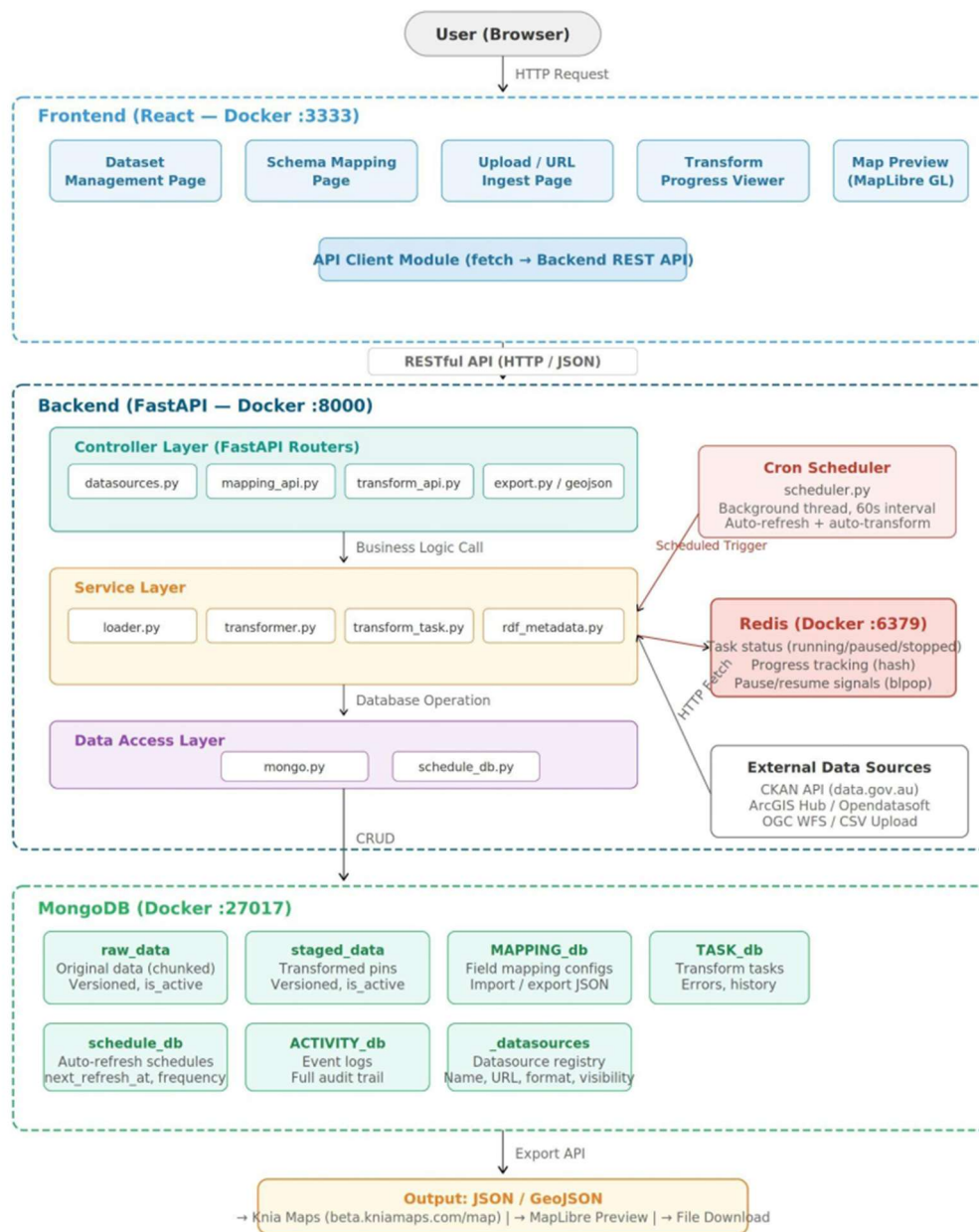


Figure 1 System Architecture

Compared to the previous architecture diagram, new components have been added: A Redis container to obtain signals for the status of transformation tasks, progress tracking, and pause/resume operations, A MapLibre GL map preview page. In the main file, an automatic scheduled background thread has been added to perform periodic updates, checking the status every 60 seconds and automatically

performing transformation when the mapping file exists.

3. Design Justifications

3.1 How Project Requirements Evolved Over Sprints

Sprint 1: Initial Proposal

Our proposal started with a simple ETL pipeline with manual data upload, schema mapping, and transformation. The architecture was originally composed of three containers (Frontend, Backend, MongoDB) and presupposed a simple workflow: upload a file, map fields, transform, and export.

Sprint 2: Evolved Requirements

Based on the experience of development and client feedback, we had to make a number of new requirements that dramatically altered our design:

Automated URL ingestion: The client (Nikhil / Knia Maps team) asked that rather than just accepting files uploading, URLs to a dataset page should be accepted and the system would automatically generate the download link. This resulted in the creation of a multi-platform resolver that served CKAN, ArcGIS Hub, Opendatasoft, OGC WFS and direct download links.

Scheduled auto-refresh: The client wanted to have automated weekly updates in data. This was not in the initial plan and needed to include the addition of a background scheduler, a schedule database and the notion of frequency-based re-ingestion.

Version control and rollback: In testing, we found that the automated updates would

create bad data without notice. The client also affirmed that there is a need to have version tracking and the capability to roll back to the earlier version immediately. This motivated the architecture of the system of flags of the `is_active` in both the `raw_data` and `staged_data` databases.

Redis as a task management system: Initially, we were going to monitor the progress of transforms by polling MongoDB. When implementing, we discovered that MongoDB polling was too slow to keep progress updated in real time and pause/resume needed a more responsive system. This was the birth of Redis as a state management of tasks.

RDF metadata extraction: The client specifically asked the system to extract metadata (description, license, tags) of data source platforms as well as of the raw data. This included the RDF metadata extraction module.

GeoJSON and map preview: Geo spatial map preview was optional in the project brief but was a priority by the client in Sprint 2. We implemented a GeoJSON API endpoint and MapLibre GL JS in the frontend.

3.2 Design Iterations and Justifications

3.2.1 Task State Management: MongoDB Polling → Redis

Original design: After every chunk was processed, status updates to MongoDB were written down to keep track of the design progress. The frontend had a polling rate of one second to a MongoDB-supported endpoint.

What has changed: The transform took seconds when using datasets with 60,000+ records. The polling rate of the frontend was so low that it could not capture the

progress in-between, and MongoDB adds redundant database load to every chunk to add.

What we have done: Added Redis as a dedicated task state store. Status (running, paused, stopped) and progress (total, processed, error count) are Redis keys and hashes. Pause relies on Redis blpop, a blocking operation that does not use any CPU until it is resumed. Once the task is finished, Redis keys expire automatically after 30 seconds (via EXPIRE with TTL) to allow the frontend time to access the final state before cleaning up.

Why this is more efficient: Redis can serve sub-millisecond reads to do progress polling, blpop removes busy-waiting during pause and the TTL-based cleanup can ensure that stale keys are not accumulated. Persistent task history is still done in MongoDB, but Redis is used on the real-time layer.

3.2.2 Data Storage: Delete-and-Replace → Version Preservation

Original design: Each new data import would remove the data in both raw data and staged data of the previous version and add new records.

What has changed: The client demanded rollback. Deletion of data cannot be undone, unless it is re-downloaded and re-transformed, which requires that external APIs are present and have not changed.

What we have done: Switched to a version-based system, with each import increasing a version number. The old ones are denoted by `is_active: false` but they do not get deleted. The version numbers of both raw and staged data are synchronized. Rollback merely switches both databases back and forth on the `is_active` flags.

Why this is more efficient: Rollback is immediate (one MongoDB update many per database) and doesn't need to be re-processed. In the case of a 24,000-record dataset, each version takes about 1.7MB - 10 versions would be stored in 17MB which is insignificant. It is evident that the trade-off between storage space and instant rollback and data protection is valuable.

3.2.3 Mapping Format: Hardcoded Transformer → Configurable field_mappings

Original design: The transformer.py was designed to use a hardcoded mapping format (static_map, accessibility_map, defaults) that had to be written manually per data source.

What has changed: The frontend team created a visual mapping interface in which users create relationships between source and target fields. This UI automatically generates a format of field_mappings (array) (source_field to target_field), which was not in the required format of the transformer.

What we have done: We did not rewrite the entire transformer (which we had already tested and verified), but instead we added in the transformer a function called `convert_mapping_format()` in the file transform_task.py that transforms the array of field mappings in the frontend into the structure of the transformer at runtime: static_map / accessibility_map / defaults.

Why this is more efficient: This method retained the working logic of the transformer but allowed the new UI-based mapping workflow. The conversion layer is an adapter pattern that separates the data model of the frontend and processing logic of the backend. When one or both sides are changed, it only requires the adapter to change.

3.2.4 URL Ingestion: Single Platform → Multi-Platform Resolver Chain

Original design: The first proposal presupposed that the data could be uploaded in the form of files. The feature of URL ingestion was introduced and only CKAN API was supported.

What has changed: The testing on real government data sources in Australia showed that various states employed various platforms: data.gov.au employed CKAN, City of Sydney employed ArcGIS Hub, Melbourne employed Opendatasoft, and some sources provide only OGC WFS endpoints.

What we have done: We created a resolver chain in the file, `url ingest.py`, where we sequentially test the following platforms: CKAN API, Opendatasoft, HTML scraping, ArcGIS Hub, OGC WFS. All resolvers are independent functions (`try_ckan`, `try_opendatasoft`, `try_scrape`, `try_arcgis`, `try_wfs`) that would download a URL and format on successful completion, or return `None` to the next resolver.

Why this is more efficient: The chain pattern is extensible - to add a new platform, you need to write a single function and add it to the chain. The sorting is by frequency: CKAN is attempted first since the majority of portals offered by Australian governments use it, and to reduce unwanted network requests.

3.2.5 Export: Single Format → Dual Mode with Pagination

Original design: The export endpoint sent all transformed pins in a single response in JSON format without pagination.

What has changed: The response payload was too big to serve the browsers effectively due to 24,000+ records per dataset and having multiple datasets. Another

feature demanded by the frontend team was to have the possibility to export raw (untransformed) data to debug.

What we have done: We added pagination (the default 500 records per page query parameter) and a 'type' parameter (raw or transformed) to the same endpoint. Raw export reads chunked data, similar to raw data; transformed export reads active pins, filtered by `is_active`, of the `staged_data`.

Why this is more efficient: This endpoint satisfies both requirements and allows the API surface to be kept low. Pagination avoids memory problems both in the server and the client. The filter is used to guarantee that exports are always the latest version and rollback can be immediately seen in the exported data.

3.3 Complex Algorithms and Implementation Details

3.3.1 Multi-Format Data Normalisation

The transformation of raw input to normalized form is the task of the function `normalize_raw_input` in `transformer.py`, which processes three different data types and gives a consistent output:

1. ToiletMap format (fields + records arrays): each field is zipped with record arrays to create a list of dictionaries.
2. GeoJSON FeatureCollection: Parses the features array out.
3. Flat list of dictionaries: Does not change.

A recursive call to the function `flatten_record` is then used to flatten nested structures, but with special treatment of GeoJSON properties (which are promoted to first level) and `geometry.coordinates` (which are treated as 0 and 1). This two-step normalisation

will guarantee that no matter the input format, the mapper will be presented with a flat dictionary with the same key paths.

3.3.2 Chunked Processing with State Control

The transform pipeline reads the data in 1,000-record blocks (the same size as the storage chunk) and uses Redis to control the state:

1. Read a chunk of raw data in `raw_data`.
2. Redis: `status task:{id}:status`.
3. “paused: block on `blpop(task:{id}:signal)` - no CPU used, wakes up on `resume/stop`.
4. When stopped: continue marching to MongoDB, clean up, and quit.
5. Condition: Running: normalise records, map records, batch-insert 500 pins to staged data.
6. Reset Redis progress hash.
7. Repeat on following chunk.

There is no error handling: a record which fails is recorded to the errors array in `TASK_db` but does not stop processing of the rest of the records. This guarantees optimal data throughput despite the imperfect source data.

3.3.3 Version-Synchronised Rollback

Rollback can work on two databases at the same time:

```
raw_data[provider].update_many({is_active: true}, {$set: {is_active: false}})
```

```
raw_data[provider].update_many({version: target}, {$set: {is_active: true}})
```

```
staged_data[provider].update_many({is_active: true}, {$set: {is_active: false}})
```

```
staged_data[provider].update_many({version: target}, {$set: {is_active: true}})
```

This represents four MongoDB operations, all in indexed fields (`is\_active` and version), with large collections taking milliseconds. The export endpoint filter of `{is_active: true}` implies the rollback will be effective immediately - no need to invalidate the cache or reprocess.

4. User-Driven Evaluation of Solution

4.1 Main Features of the Solution

4.1.1 Data Ingestion

The system is aided by two ingestion modes. URL ingestion enables users to copy a dataset page URL of any of the supported platforms (CKAN, ArcGIS Hub, Opendatasoft, OGC WFS). The backend automatically identifies the type of platform, resolves the download link, downloads and processes the data (JSON, GeoJSON, CSV, ZIP) and retrieves RDF metadata (description, license, tags) and stores the data in versioned blocks of 1,000 records each. File upload enables users to directly upload JSON, GeoJSON, or CSV files using the same chunking and versioning logic. Each of the two modes registers the datasource to be managed and can also be configured to automatically refresh itself every week.

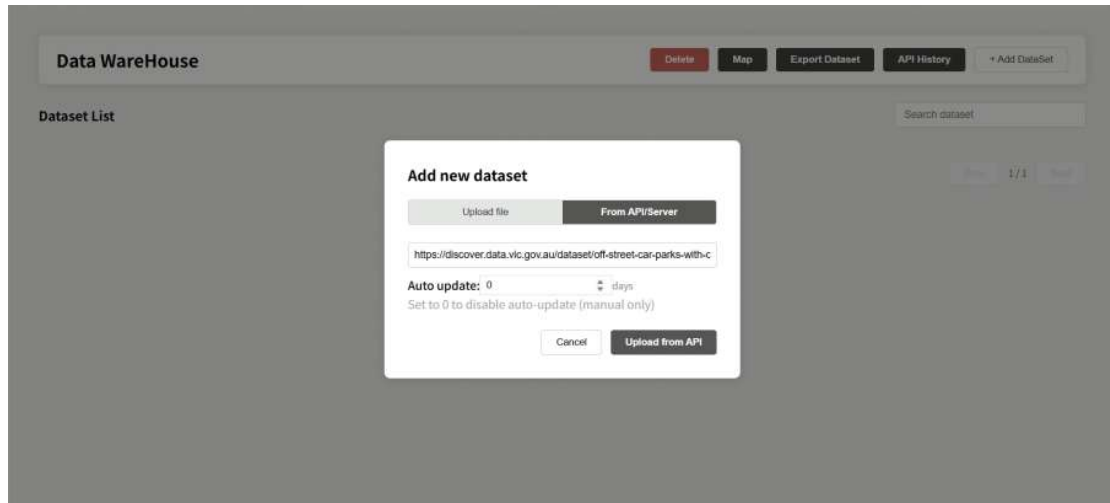


Figure 2 URL ingest page with platform auto-detection

4.1.2 Schema Mapping

The mapping interface shows source fields (automatically extracted by the data ingested) on the left and standard pin schema fields on the right. Users make associations among similar fields, such as between the latitude and geometry.coordinates[0]. The mapping configuration is stored within MongoDB and may be backed up or shared as a JSON file, and imported into other environments. Version-specific queries can be made on source fields, enabling users to see field differences between data versions.

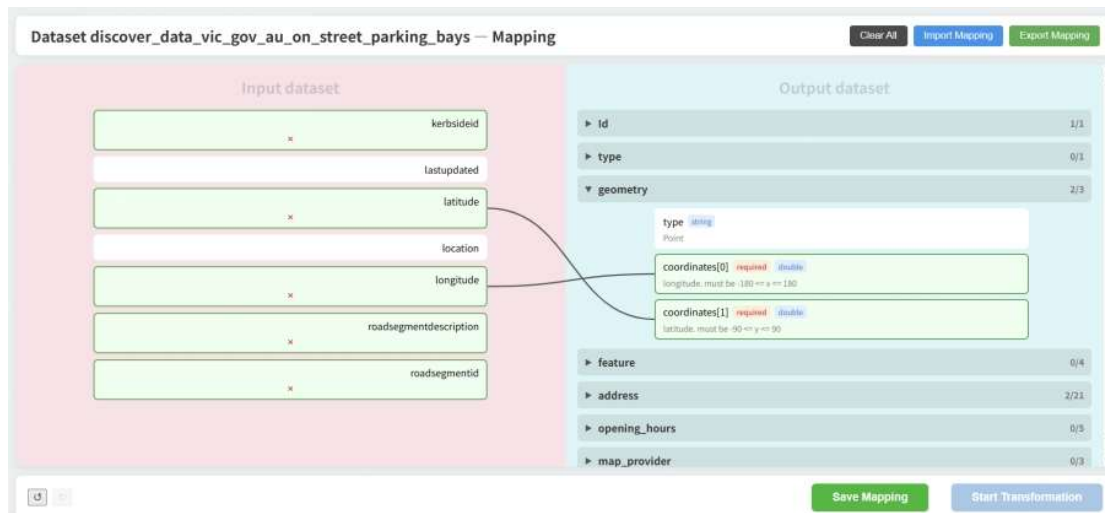


Figure 3 Mapping page with source-target field connections

4.1.3 Data Transformation

The transform pipeline reads the raw data in chunks, performs the mapping configuration, and pushes standardised pins to the staged database. The whole process is operated as a background process and has real time controls: Start, Pause, Resume, and Stop. The tracking of progress is made through Redis and the frontend can show a dynamic progress bar (e.g., 12,000 / 23,864). Mistakes are identified on a record basis and recorded without halting the entire task. All transform outputs are marked with a version number and an is-active flag to roll back.

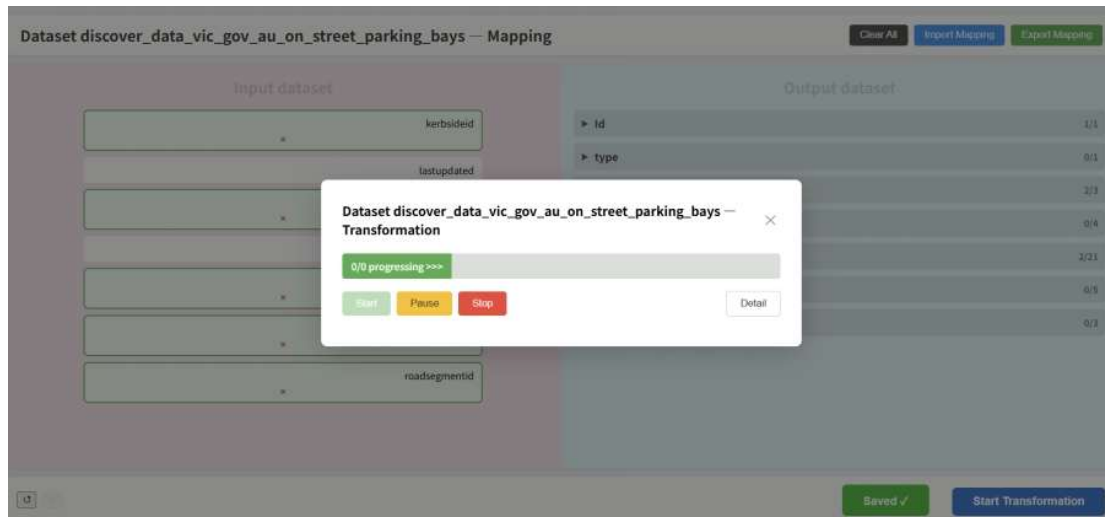


Figure 4 Transform progress dialog with Start/Pause/Stop buttons

4.1.4 Automated Scheduling

Data sources that are ingested by URL are automatically scheduled to be refreshed each week. Background scheduler examines all the schedules that are overdue after every 60 seconds. Once it is activated, it will re-download data using the original URL and save it as a new version and in case a mapping configuration is present, the transform will be automatically run. The festival-goers can also configure frequency days to 0 to turn off automatic updates on particular sources.

4.1.5 Version Control and Rollback

Any import of data produces a new version. Old versions are not deleted but set to have `is_active` flags. All versions with timestamps are shown on the version history page. Any version can be changed to the raw or staged database by simply clicking Roll Back on any version and both databases are automatically changed to that version, without having to re-process. The version rolled back is immediately reflected on the export endpoint.

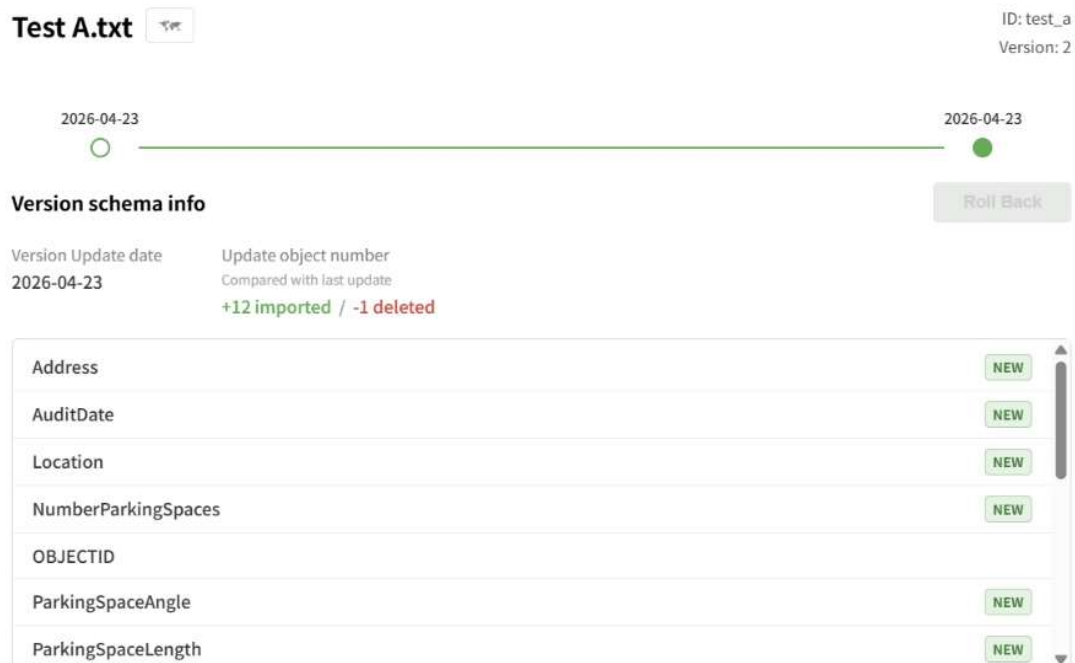


Figure 5 Version timeline with rollback button

4.1.6 Data Export

The export system provides paginated downloads of transformed pin data, and raw source data, using a single endpoint, with a type parameter. Various datasets of choice are available as individual files. The transformed pin data is fed to a MapLibre GL JS map preview using a GeoJSON endpoint and when a user clicks on a point, their properties are displayed in a popup.

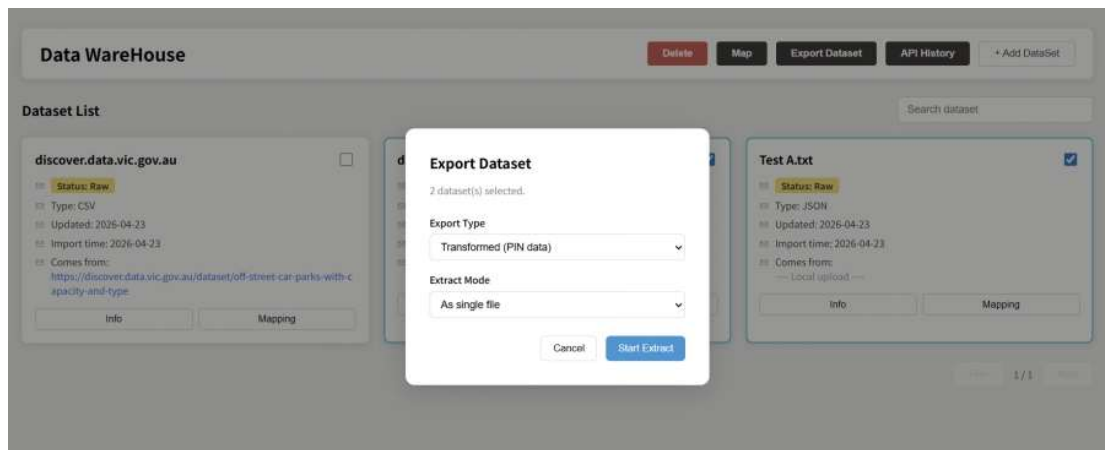


Figure 6 Export page

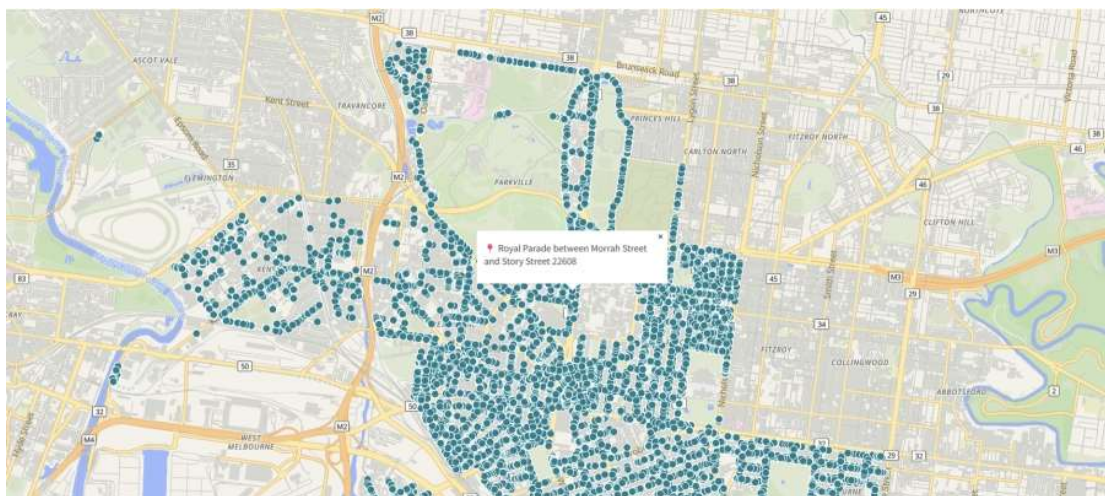


Figure 7 map preview with clickable pins

4.1.7 Datasource Management

The list of registered data sources on the main page includes the status, type, date of last update, the number of pins, and the source URL. Users are able to see all the details, clone a datasource set-up, make it visible to end users or destroy datasource. Metadata endpoint is a source of RDF-extracted information (description, license, publisher, tags) about the source. Activity events are logged in and give a complete audit trail of all operations.

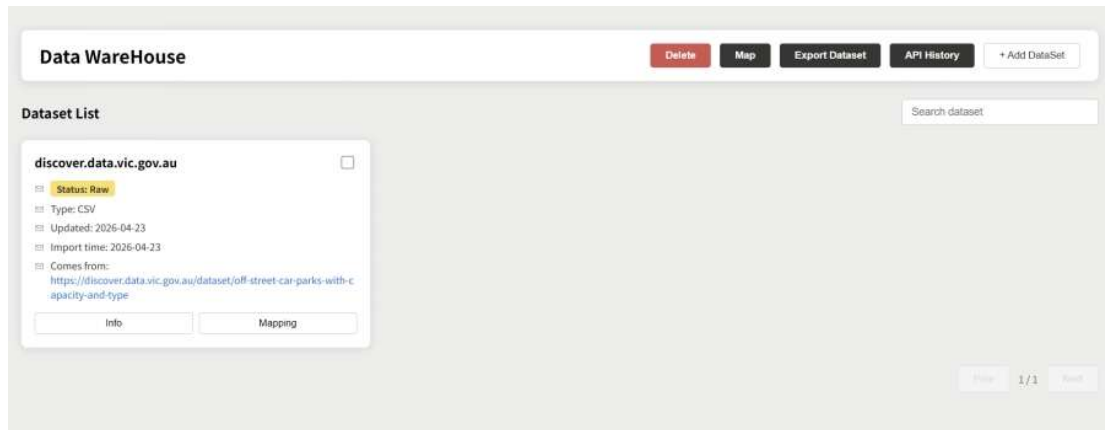


Figure 8 Dataset list page with Info/Mapping/Duplicate buttons

4.2 Evaluation Criteria

To determine the effectiveness of our solution to both the client (Knia Maps team) and the target end users (administrators who will handle accessibility data), we set the following criteria:

#	Criterion	Description	Target
E1	Multi-source ingestion	System can ingest data from different government platforms and file formats	Support at least 5 platform types and 3 file formats
E2	Schema transformation accuracy	Transformed pins correctly map source fields to the standard pin schema	100% of mapped fields populate correctly, no data loss
E3	Large dataset handling	System handles datasets with tens of thousands of records without failure	Process 60,000+ records with zero crashes

E4	Transform control	Users can start, pause, resume, and stop transforms with real-time feedback	All four controls functional, progress updates within 2 seconds
E5	Automated freshness	Data sources refresh automatically at configured intervals	Scheduler triggers within 60 seconds of due time
E6	Rollback reliability	Version rollback switches data instantly without re-processing	Rollback completes in under 1 second for any dataset size
E7	Export completeness	Exported data includes all transformed fields and supports both raw and transformed modes	Exported JSON matches staged_db content exactly
E8	Map visualisation	Transformed pins render on a map with clickable detail popups	All pins with valid coordinates display correctly
E9	System reliability	The system runs continuously without manual intervention	No unhandled crashes over a 7-day period
E10	Deployment portability	The entire system can be deployed on any machine with Docker	Single `docker compose up` brings up all services

4.3 Evaluation of Effectiveness

Criteria Met

E1 — Multi-source ingestion: ACHIEVED. The resolver chain allows 5 platform types (CKAN, ArcGIS Hub, Opendatasoft, OGC WFS, direct links) and 3 or more file formats (JSON, GeoJSON, CSV, ZIP). Known to work with 7 real Australian government data sources such as data.gov.au (CKAN), data.cityofsydney.nsw.gov.au (ArcGIS), discover.data.vic.gov.au (CKAN), and data.sa.gov.au (CKAN).

E2 — Schema transformation accuracy: ACHIEVED. Validated by comparing staged_db output with known pin schema. Fields of type (Id, coordinates, type) fill out right. The adapter, which is used to translate the frontend field mappings format to the internal format of the transformer, is the correct one, which is the adapter called as the convert mapping format.

E3 — Large dataset handling: ACHIEVED. Managed to successfully process the Melbourne On-Street Parking Bays data (23,864 records) and the National Public Toilet Map (65,578 records) with 0 crashes. Chunked storage and batch inserts maintain the same amount of memory use irrespective of the size of the dataset.

E4 — Transform control: ACHIEVED. Start, Pause, Resume, and Stop all function correctly.

E5 — Automated freshness: ACHIEVED. The scheduler executes every 60 seconds, properly detects overdue schedules and initiates re-ingestion. In event of a mapping configuration, transformation is automatically activated following ingestion. A correct setting of frequency_days to 0 is such that it turns off auto-refresh.

E6 — Rollback reliability: ACHIEVED. Rollback sets is_active flags on both the raw and staged data. Swapped version between v1 and v2 of the parking bays dataset - export output updated instantly. The operation takes a few milliseconds irrespective

of size of the dataset.

E7 — Export completeness: ACHIEVED. Export supports pagination and type=transformed and type=raw. JSON that is exported corresponds to staged_db. Various datasets are available as individual files. GeoJSON endpoint properly formats data to be used in MapLibre.

E8 — Map visualisation: ACHIEVED. MapLibre GL JS will render all pins that have valid coordinates. When a pin is clicked, a popup with non-null properties is displayed. The GeoJSON endpoint properly parses the coordinates of strings into floats and omits the record with incorrect or missing coordinates.

E10 — Deployment portability: ACHIEVED. The whole system can be started up with docker compose up -build. MacOS (development) tested. The five containers (Frontend, Backend, MongoDB, Redis, Preview) are all started correctly and with the correct networking.

Criteria Not Fully Met

E9 — System reliability (7-day continuous run): We have not done a complete 7 day continuous system reliability test. We could not carry out this test due to the time constraints and equipment limitations. Scheduler and transform tasks also have error processing and isolation (per-record and per-provider) but we cannot assure zero unhandled exceptions in the long term. Known risk: when Redis is no longer available during a running transform, the thread that is executing the transform will crash. Connection retry logic would enhance resilience by adding Redis connection retry logic.

5. Limitations and future work

5.1 Map preview performance limitations

At present, the system has certain performance problems when preview the map at the full data warehouse level.

When it is necessary for display multiple data sets at same time, due to the large numbers of geographical data points, the front-end use MapLibre GL JS to render slow loading speed and not smooth interaction response, such as delay when scaling, dragging or clicking points.

This problem is mainly caused by the rendering overhead of large-scale GeoJSON data.

Therefore, we recommend that user gives priority to the map preview of a single data set, and only use the global preview function when they need to analyze the spatial relationship between different data sets.

Future improvement direction:

- Introduces point clustering to reduce rendering pressures
- Optimizing GeoJSON data size and transmission efficiency

5.2 Automatic update configuration is not flexible enough

At present, the automatic update function of the system has certain limitations in terms of configuration.

If user want to modify the automatic update cycle of the data set, it must be achieved by importing again the data, which is tedious and not intuitive enough. In addition, the current API History page only supports viewing and deleting scheduling tasks, and cannot directly edit the existing schedule.

The design reduces the ease of use of the system to a certain extent.

Future improvement direction:

- Add schedule editing function in the API History page
- Support direct modification of update frequency and status
- Provide clearer user interface feedbacks to improve the interactive experience

5.3 Lack of independent Mapping configuration management page

Although the current system supports importing and exporting mapping configurations in the Mapping page, it has lacks an independent configuration management interface.

Users cannot conveniently manage exist mapping configurations, such as viewing, editing, reusing or comparing different versions. This will increase the workload of duplicate configuration and reduce efficiency when processing multiple data sets with similar structures.

Future improvement direction:

- Design an independent Mapping management page
- Support version management and reuse of mapping configuration
- Provide visual field mapping editing tools

5.4 Description Limited rendering ability

The system can currently extract description and license information from the RDF metadata and display it on the Information page. At same time, the system can correct and parse and render the HTML content contained in the description.

However, at the present, only HTML format is supported, and other common formats such as Markdown are not support for the time being, resulting in the descriptive information of some data sources that can only be displayed in the plain text form, it affects readability and display effect.

Future improvement direction:

- Increase support for parsing and rendering of Markdown format
- Optimize text style and typesetting to improve the reading experience
- Support more types of metadata display formats

6. Project Handover

Handover Steps

In a bid to make the project smoother once handed over to our client, we carried out the following handover steps:

1. Documentation preparation: We have prepared detailed documentation, such as an Installation Manual (PDF) with all environment requirements, Docker installation guidelines, and step-by-step deployment guidelines to run the system on our machines.
2. Packaging of source code: The entire project source code was stuffed into a deliverable archive file (capstone-project-26t1-9900-t14a-bread.zip) with all the backend, frontend, database configuration and Docker Compose files required to build and run the entire system.
3. Handover email: A formal email of handover was sent to the client (Nick Rabey) on

27 April 2026, and all team members were included. Two important features of the email were the two attachments that were the source code of the project (in compressed form) and the Installation and User Guide (PD) document containing the information on the system set up and the running of the system.

4. Handover content summary: The deliverables to be provided to the client are:

- Full source code Docker containerised (Frontend, Backend, MongoDB, Redis)
- pipeline that supports various government data platforms (CKAN, ArcGIS Hub, Opendatasoft, OGC WFS)
- Schema mapping and transformation system with start/pause/stop
- Automated weekly data refresh system.
- Version control with immediate rollback option.
- JSON and GeoJSON export of data.
- Deployment instructions in the form of an Installation Manual.

5. Ongoing support offer: During the handover email, we advised that we would be available to provide assistance in case of any queries about the system set up or usage and so that the client would have a point of contact after the handover.

Evidence of Handover Communication:

